

Unbalanced data processing for software defect prediction

Yang Qu

Information Science and
Technology College
Dalian Maritime University
Dalian, China
2361083688@qq.com

Zhenming Li

Information Science and
Technology College
Dalian Maritime University
Dalian, China
917681743@qq.com

Jiaoru Zhao

Information Science and
Technology College
Dalian Maritime University
Dalian, China
zjr@dmlu.edu.cn

Hui Li

Information Science and
Technology College
Dalian Maritime University
Dalian, China
li_hui@dmlu.edu.cn

Abstract—How to solve the imbalance of defect classification in software defect prediction and improve the accuracy of prediction is an important problem in software testing. Thus many machining learning based model, such as self-adaptive Robust Synthetic Minority Over-Sampling Technique (RSMOTE), were presented for software defect prediction. However, the imbalanced data distribution limited the prediction performance. Addressing to this issue, a RSMOTE-based Data Imbalance Processing (RDIP) model is presented in this paper. Specifically, the normalized outlier data is removed according to the European distance between points in data denoising, and then the fuzzy membership and fuzzy labels of each point are calculated using the Computational Class Fuzzy Algorithm (FCMD), which removes the hazard points and noise points according to the selection boundary point algorithm (BRS). Experimental results the date sets of NASA, Promise show that the average F1-measure of software defect prediction method for data imbalance is 6.98% higher than other comparison algorithms, which can effectively solve the problem of defect classification imbalance in software defect prediction.

Keywords—Software defect prediction, Data imbalanced, Cross project defect prediction, Fuzzy entropy

I. INTRODUCTION

The current times are one of rapid development of information technology and tools. Software systems have long been used in a wide range of industries. In the software life cycle, every process inevitably generates errors and various types of faults. Actually, software systems without errors or faults cannot exist in a real-life production environment.

Software defects refer to the error introduced in the development process of the software system. Software defect will lead to the error, failure and collapse of the software systems during operation and even endanger the safety of human life and property [1]. At present, the studies of software defect prediction mainly focus on the following aspects. First, the predictive model can be used for quality assurance before product release. However, it would be more helpful if defects can be predicted when the source code is changed. Secondly, for new projects or little historical data, it is not possible to build models from historical data. With the use of process indicators becoming more and more popular, this limitation has become one of the most difficult problems in software defect prediction research [2]. With the use of process

indicators becoming more and more popular, this limitation has become one of the most difficult problems in software defect prediction research [2]. To solve this problem, researchers have proposed a variety of cross project defect prediction models [3-4].

In software defect prediction, the traditional classification algorithm does not deal with the datasets, directly constructs the model and makes prediction. However, the distribution of software defects in software modules is unbalanced. Conventional classification algorithms cannot achieve a good effect in the classification of unbalanced datasets, but they can achieve a fairly good effect in the classification of relatively balanced datasets. Therefore, the classification algorithm is more inclined to defect free modules, and the accuracy of predicting defective modules will be greatly reduced [5-6]. Therefore, how to solve the problem of unbalanced datasets is the key to the success or failure of software defect prediction.

There are many models for dealing with classification imbalance. The commonly used models include stochastic over-sampling, stochastic under-sampling and self-adaptive Robust Synthetic Minority Over-Sampling Technique (RSMOTE) [7]. However, the process of random oversampling can lead to a lack of generalization and overfitting of the classification model to a few classes. Random undersampling will remove most of the samples in the class to achieve category balance, which will also lose more sample information. RSMOTE also has uncontrolled quality issue of the simulation samples due to edge points. To address this issue, we present the a RSMOTE-based Data Imbalance Processing (RDIP) model, which combines the fuzzy entropy and RSMOTE algorithms to control and improve the quality of the generated simulation samples, thus improving the prediction accuracy.

The main research content of this paper has the following two points:

- To overcome the classification problem by imbalanced data distribution, we present the RDIP model, which combines the fuzzy entropy and RSMOTE to control and improve the quality of the generated simulation samples, thus improving the prediction accuracy.
- RDIP is applied to cross project prediction. According to the experimental results, this method can improve the

class imbalance problem in cross project prediction. Compared with the original method without any imbalance treatment strategy, RDIP can improve the prediction performance.

II. RELATED WORK

In order to solve the problem of imbalance, sampling method is used to change datasets distribution, which includes mixed sampling, over-sampling and under-sampling. Khoshgoftaar et al. [8] uses a new processing method, which combines feature selection and random undersampling in processing relevant data. In order to improve the total number of minority samples, Kaminsky et al. [9] adopted the method of copying minority samples, which is more convenient and convenient for sampling. Guo et al. [10] changed the range for generating simulation samples from a straight line between two points to a spatial range with two points as diagonals and proposed two constraints, which solved the problems of the lack of spatial generalization ability of SMOTE and the poor performance of the SMOTE algorithm on the "convex" distribution of data.

Cost-sensitive classification algorithms can be modified to predict defective modules with higher accuracy, thus solving the imbalance problem. In 2009, Moser et al. [11] introduced cost-sensitive learning method into software defect prediction and achieved good results. Siers et al. [12] proposed a cost-sensitive voting method and decision-making forest tree. In 2012, Miao et al. [13] proposed a defect prediction model of cost-sensitive neural network, and utilized cost-sensitive neural network to optimize it. Experiments proved that the effect of the proposed model was better than traditional neural network. In 2015, Omer [14] used cost-sensitive neural network combined with artificial bee colony algorithm to predict software defects. These models achieved good performance on software defect predictions.

III. RDIP MODEL

A. Model Overview

Fig.1 shows the flow of RDIP model, which is briefly introduced below. The model construction is mainly completed by four steps. The first step, we calculate the Euclidean distance between each point and all other points in the datasets, sort out the five smallest distances and calculate the average distance of these five distances. We remove the top-5 with the largest average distance. The second step, we use Fuzzy Class Membership Degree value algorithm (FCMD) to calculate the fuzzy membership degree and fuzzy label of each point. The third step, we use Bug Reports Selection (BRS) to remove danger and noise points. The fourth step, we use RSMOTE to generate the simulation sample, add the simulation sample to the training sets, determine whether the test accuracy will be improved.

B. De-noising Module

We divide the data into four categories: safe point, outlier point, dangerous point and noise point. We hope to remove the last three kinds of noise data, and the remaining safe point data will be used to generate simulation samples.

Fig.2 is a schematic diagram of dangerous points, safe points and noise points. Circles represent the majority of

categories and stars represent the minority. In the Fig 2, among the six closest points to point B, five are of the same type and one is of a different type. Then point B is considered safe and can be retained. Among the six nearest points to point C, there are no points of the same type. Then point C is considered to be a noisy point and should be deleted. Among the 6 nearest points to point A, there are 2 points of the same category and 4 points of different categories. In such a situation where the proportion of points in the two categories among the nearest K points is similar, point A is considered to be a dangerous point. It is necessary to determine which category A by calculating fuzzy entropy and fuzzy affiliation, and further determine whether it will be deleted or retained.

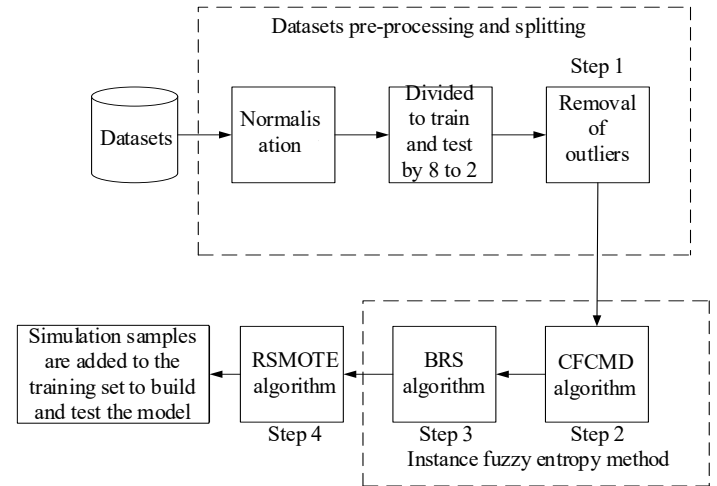


Fig. 1. The framework of RDIP

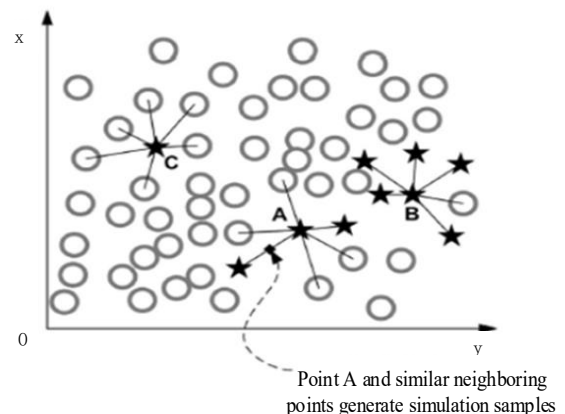


Fig. 2. Schematic diagram of dangerous points, safety points and noise points

The schematic diagram of outlier points is shown in Fig.3. Triangular points represent minority class points and circular points represent majority class points. Although red triangle point D adjacent points and point D are the minority class point. According to the previous definition, D should be safer. But noted that the point and neighboring points distance too far, we think this point, even if it meets the requirement of safe points must also be removed, otherwise generated simulation samples may be more inclined to the majority of the class. We define points like D as outlier points.

We calculate the distance between each point x_i and other points. After sorting, we take the 5 points with the smallest distance and calculate their average value. $avr(X_i)$ is recorded as the average distance between this point x_i and other points, and we sort them in descending order and delete the top-5 outliers. $avr(X_i)$ is expressed as

$$avr(X_i) = avr(Dis(X_i, X_1) + Dis(X_i, X_2) + \dots + Dis(X_i, X_5)), \quad (1)$$

where $Dis(X_i, X_1)$ is the Euclidean distance between X_i and X_1 , and $X_1 \sim X_5$ is the 5 nearest points X_i .

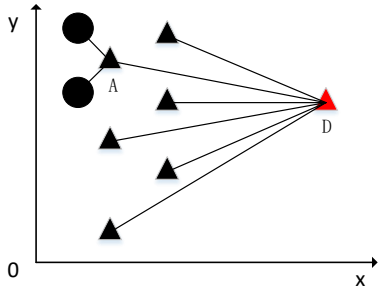


Fig. 3. Schematic diagram of outlier points

C. Calculate FCMD

In step 1, for any point $x_i (1 \leq i \leq n)$, n is the amount of data in the datasets. Adjacent point parameter (K_0) points closest to the Euclidean formula are selected from the training sets DT , denoted as $x_i^{K_0} = \{x_i^1, \dots, x_i^k, \dots, x_i^{K_0}\}$. The Euclidean distance at any point between x_i and $x_i^{K_0}$ is named as E_i^k . In Step 2, we calculate the number of points in each category C_j in $x_i^{K_0}$. The value of j is not less than 1 and the value of j is not greater than P , where P represents the number of categories. Meanwhile, we calculate the number of points with the same number, and then use formula (2) to calculate the membership value of each category $u_{C_j}(x_i)$.

$$u_{C_j}(x_i) = \begin{cases} 0.51 + (\frac{n_i}{K_0}) * 0.49, & C(x_i) = C_j \\ (\frac{n_i}{K_0}) * 0.49, & C(x_i) \neq C_j \end{cases}, \quad (2)$$

In Step 3, we use formula (3) to calculate the categories of $u_{C_j}(x_i)$ and E_i^k , and calculate point x_i belonging to the fuzzy membership value of class C_j .

$$FCM(x_i, C_j) = \frac{\sum_{k=1}^{K_0} u_{C_j}(x_i) \left(\frac{1}{\|x_i - x_i^k\|^{\frac{2}{m-1}}} \right)}{\sum_{k=1}^{K_0} \left(\frac{1}{\|x_i - x_i^k\|^{\frac{2}{m-1}}} \right)}, \quad (3)$$

where $\|x_i - x_i^k\|$ represents the Euclidean distance between points x_i and E_i^k . m is the weight parameter that determines the influence of the value of $\|x_i - x_i^k\|$ on the calculation of

membership. In this model, we set its value as 2. In Step 4, according to $FCM(x_i, C_j)$, calculate the fuzzy category label of point x_i , denoted as $C_F(x_i)$, and its formula is formula (4),

$$C_F(x_i) = \operatorname{argmax}(FCM(x_i, C_j)), \quad (4)$$

Through the above four steps, we can calculate the fuzzy membership degree and fuzzy label of each point.

D. BRS remove danger points and noise points

BRS algorithm process is shown in Algorithm 2. The BRS algorithm has four inputs: the fuzzy class membership value of the point $FCM(x_i, C_j)$, the fuzzy class label of the point $C_F(x_i)$, K_0 parameter represents the number of the nearest neighbors, and α parameter represents the fuzzy entropy threshold. In this article, α parameter is set to 0.7.

Algorithm 2 BRS algorithm

Input: Fuzzy membership $FCM(x_i, C_j)$, fuzzy label $C_F(x_i)$, parameter K_0 , fuzzy entropy a .

Output: Removed training sets with danger points and noise points T'

- 1: Set an empty set T'
- 2: According to Formula (1), calculate a minority class point x_i the fuzzy entropy.
 - a) If x_i fuzzy entropy $< a$, add x_i to T' .
 - b) Ignore x_i .
- 3: For each x_i , find K_0 neighboring points $x_i^{K_0} = \{x_i^1, \dots, x_i^k, \dots, x_i^{K_0}\}$.
 - a) If points in $x_i^{K_0}$ belong to the same class as x_i , add them to T' .
 - b) Ignore x_i .
- 4: For each x_i , its fuzzy label is compared to the real label.
 - a) If they are the same, add x_i to T' .
 - b) Ignore x_i .
- 5: Return T' .

E. RSMOTE generate simulation samples

RSMOTE mainly improves the insufficient spatial generalization ability of SMOTE and the poor performance of data "convex" distribution. The advantage of RSMOTE over SMOTE is that instead of simply taking the point on the line between two samples as the SMOTE sample. RSMOTE specifies the value range of a SMOTE sample in the space and get the SMOTE sample in this space. In this way, the samples taken out have better spatial generalization ability. For the convex case proposed two constraints to avoid the noise generated by the convex case.

F. RDIP predicts software defects

The framework of RDIP is shown in Fig 1, and the algorithm execution process is shown in Algorithm 4. The first 80% of the normalized datasets was taken as the training sets, and the last 20% of the datasets was taken as the test sets.

The training sets were processed in four steps as follows, and the simulation samples were finally obtained. The simulation samples were added into the training sets to build the model.

Algorithm 4 RDIP algorithm

Input: *Normalized training sets T*

Output: *balance sample sets*

- 1: Calculate every point $avr(X_i)$, and delete the 5 largest points of $avr(X_i)$ in the training sets T.
 - 2: The training sets T with outlier points removed is input into FCMD and BRS algorithms, and the training sets T' with danger points and noise points removed is obtained.
 - 3: Enter the training sets T' with rid of outliers, noise and dangerous point into RSMOTE.
 - 4: Returns balance samples.
-

IV. EXPERIMENT RESULTS AND ANALYSIS

A. Experimental Datasets

NASAMDP and Promise are widely used in the experiment of software defect prediction. In our experiment, we used 20 datasets from NASAMDP and Promise. In TABLE I, datasets PC1, PC11, PC3, PC4, KC1 and CM1 belong to NASAMDP, and other datasets belong to Promise public datasets.

TABLE I. DETAIL OF EACH DATASETS

Name	Number of metrics	Number of most categories	Number of minority categories	Imbalance rate
Ivy 2.0	20	312	40	11.36%
PC4	37	1110	177	13.75%
Velocity 1.4	20	147	49	25.00%
Camel 1.2	20	392	216	35.53%
CM1	37	285	42	12.84%
Ivy	20	312	40	11.36%
Jedit 3.2	20	182	90	33.09%
Jedit 4.2	20	319	48	13.08%
Log4j 1.1	20	72	37	33.94%
PC1	37	644	61	8.65%
PC3	37	943	134	12.44%
PC11	21	1032	77	6.94%
Synapse 1.1	20	162	60	27.03%
Jedit 4	20	231	75	24.51%
KC1	21	1783	326	15.46%
Lucenedata	17	627	64	9.26%
Poi 2.5	20	248	137	35.58%
Synapse 1.0	20	141	16	10.19%
Velocity 1.5	20	142	72	33.64%
Jedit 4.1	20	233	79	25.32%

B. Evaluation Standard

In this study, we used a wide range of indicators, namely Precision, Recall and F1-measure, to measure the performance of the method. FP represents the number of modules that are predicted to be defective but are in fact defect-free. FN

represents the total number of modules predicted to failure. TP represents the number of modules that are predicted to be defect-free but are in fact defective. TN represents the total number of modules predicted right.

- $$Precision = \frac{TP}{TP+FP} \quad (4.1)$$

- $$Recall = \frac{TP}{TP+FN} \quad (4.2)$$

- $$F1 - measure = \frac{2 \cdot Recall \cdot Precision}{Recall + Precision} \quad (4.3)$$

C. Experimental Result

a) Verify the RDIP method whether can improve class imbalance

In this experiment, we used RDIP method to investigate 9 METHODS of RSMOTE, ROS, RUS, SMOTE, SMB, RUS_bal, SPIDER, SPIDER+ADACOST and max-min normalised RSMOTE in J48, NBM, SVC, NB and KNN Model was built under the classifier for testing. Each case was tested five times and averaged. Reducing the impact of the randomness of the simulation samples generated by RSMOTE on the results. The experimental results show that the method introduced in this paper greatly improves F1-measure and effectively improves class imbalance. 12 of the 20 datasets showed significant improvements.

TABLE II. F1-MEASURE VALUES OBTAINED BY DIFFERENT CLASSIFIERS IN DATASETS AND PERFORMANCE IMPROVEMENT

Datasets	Classifier	F1-measure	Other methods to achieve maximum F1-measure	Maximum F1-measure improvement compared with other methods
Ivy 2.0	J48	0.5975	ROS+NBM	13.52%
PC4	SVC	0.5680	RUS+J48	1.50%
Velocity 1.4	SVC	0.8084	ROS+SVC	3.94%
Camel 1.2	SVC	0.4905	RUS+J48	3.55%
CM1	SVC	0.3599	ROS+SVC	15.17%
Ivy	J48	0.5265	RSMOTE+J48	21.16%
Jedit 3.2	KNN	0.7652	RSMOTE+KNN	4.26%
Jedit 4.2	NBM	0.4331	Normalized RSMOTE+NBM	3.94%
PC1	SVC	0.4256	Normalized RSMOTE+NBM	2.15%
PC3	SVC	0.4223	RUS+SVC	4.60%
PC11	J48	0.7328	SMOTE+KNN	13.79%
Synapse 1.1	SVC	0.6154	SPIDER+NBM	3.11%
Log4j 1.1	NB	0.8750	RUS+NBM	Equal

As shown in TABLE II, we know that in the above datasets, the F1-measure of the datasets processed by the RDIP is higher than that of all other datasets. Because while using RSMOTE to solve the problem of imbalance, we use the de-noising module to ensure the quality of the generated

simulation samples. The generated simulation samples are all near a few classes or the points whose fuzzy entropy is more inclined to a few classes, and there are no outlier simulation samples far from most points. The model constructed by adding reasonable and high-quality simulation samples to the training sets can effectively solve the imbalance problem and improve the testing accuracy. Therefore, it can be seen that the RDIP method is better than all other methods in dealing with the class imbalance problem.

b) *Validation of the RDIP method for cross project defect prediction*

The performance of RDIP in cross project prediction is discussed to determine if it is transferable. TABLE III shows the results using the Camel 2.0 dataset as test set.

To build the cross project setup, we selected Camel 2.0, Velocity 1.4, Camel 1.2, Ivy, Jedit 3.2, Jedit 4.2, Synapse 1.1, and log4j 1.1 for a total of eight datasets with the same software metric. We use one dataset as the training set and the other as the test set. We use RDIP to study different classifiers (NBM, NB, KNN, SVC, J48). More importantly, the cohesion set includes data from training set of the eight data sets in addition to the test set corresponding to the training set. The cohesion set is also used as training set to construct the model.

TABLE III. EXPERIMENTAL RESULTS OF RDIP FOR UNBALANCED DATA DISTRIBUTION OF CROSS PROJECT PREDICTION BASED ON CAMEL 2.0

Training set	Classifier	Evaluation standard	Value
Velocity 1.4	SVC	<i>Precision</i>	0.0556
		<i>Recall</i>	0.1250
		<i>F1-measure</i>	0.0769
Camel 1.2	NB	<i>Precision</i>	0.2800
		<i>Recall</i>	0.8750
		<i>F1-measure</i>	0.4242
Ivy	NBM	<i>Precision</i>	0.3810
		<i>Recall</i>	1.0000
		<i>F1-measure</i>	0.5517
Jedit 3.2	SVC	<i>Precision</i>	0.2222
		<i>Recall</i>	1.0000
		<i>F1-measure</i>	0.3636
Jedit 4.2	KNN	<i>Precision</i>	0.3810
		<i>Recall</i>	1.0000
		<i>F1-measure</i>	0.5517
Synapse 1.1	SVC	<i>Precision</i>	0.3333
		<i>Recall</i>	0.8750
		<i>F1-measure</i>	0.4828
Log4j 1.1	NB	<i>Precision</i>	0.4545
		<i>Recall</i>	0.6250
		<i>F1-measure</i>	0.5263
Cohesion	KNN	<i>Precision</i>	0.4375
		<i>Recall</i>	0.8750
		<i>F1-measure</i>	0.5833
Origin	KNN	<i>Precision</i>	0.6000
		<i>Recall</i>	0.3750
		<i>F1-measure</i>	0.4615

The experimental results show that four datasets can achieve F1-measure values above 0.5 when using irrelevant project information as the training set, and only Camel 1.2 and Jedit 3.2 have low Precision values, around 0.28 and 0.22 respectively. Only Velocity 1.4 has very small Precision, Recall and F1-measure. The results from Cohesion show that this method can compensate for the serious classification errors that result when using a specific dataset as the training set, such as the case of using Velocity 1.4 as the training set. However, at the same time compared to good results, Cohesion also results in lower values for the evaluation

criteria. Therefore, when we do not know which dataset to use as the training set will lead to better results, we can use Cohesion as a compromise. If we know which dataset works better as the training set, we use the corresponding dataset as the training set. However, comparing the results of the training and test sets of Ivy 2.0 in the original and without any imbalance strategy (Origin) we can see that more than half of the F1-measure is larger than the F1-measure in Origin. This means that using this method cross projects can improve imbalance to some extent but not better using one's own training set. Also, more than half of the datasets had larger Recall values than Origin, but none of the Precision metrics reached Origin. In summary, the cross project study using the RDIP method was able to improve the imbalance in the corresponding dataset and increase the F1-measure and Recall values to some extent, but the overall experimental results are not good enough.

V. CONCLUSION AND FUTURE WORK

This paper presents a model named RDIP, and the experimental verification shows that this imbalance treatment strategy is better than other imbalance treatment strategies. Improved prediction accuracy for a few classes makes software defect prediction more accurate. Applying this imbalance treatment to remove the three types of points (outliers, hazards and noise points) is effective. RDIP improves class imbalance when predicting cross projects. In the future work, we plan to solve the data imbalance problem by directly selecting high quality simulation samples from the pool of simulation samples generated by RDIP using different optimization methods. The advantage of RDIP selects better simulation samples while retaining complete location information between the points in the training sets. Because the best simulation samples are selected from multiple generations, it also improves the problem of high randomness of single prediction due to single generation of simulation samples.

ACKNOWLEDGMENT

This work was supported by National Natural Science Foundation of China (No.61902050), Natural Science Foundation of Liaoning Province (No.2021-MS-136), the Fundamental Research Funds for the Central Universities (No.3132019355).

REFERENCES

- [1] Gong L, Jiang S, Jiang L. Research Progress of Software Defect Prediction, *Journal of Software*, 2019, Vol.30, no.10, pp.3090-3114.
- [2] Zimmermann T, Nagappan N, Gall H, et al. Cross-project defect prediction: a large scale experiment on data vs. domain vs. process, *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM Sigsoft International Symposium on Foundations of Software Engineering*, 2009, pp.91-100.
- [3] Ma Y, Luo G, Zeng X, Chen A. Transfer learning for cross-company software defect prediction. *Information and Software Technology*, 2012, vol.54, no.3, pp.248-256.
- [4] Turhan B, Menzies T, Bener A B, Stefano J D. On the relative value of cross-company and within-company data for defect prediction. *Empirical Software Engineering*, 2009, vol.14, pp.540-578.
- [5] Hall T, Beecham S, Bowes D, Gray S D, Counsell A. *Systematic Literature Review on Fault Prediction Performance in Software*

- Engineering. IEEE Transactions on Software Engineering, 2012, vol.38, no.6, pp.1276-1304.
- [6] Arora I, Tatarwal V, Saha A. Open Issues in Software Defect Prediction. *Procedia Computer Science*, 2015, vol.46, pp.906-912.
 - [7] Pelayo L, Dick S. Applying Novel Resampling Strategies to Software Defect Prediction, *Fuzzy Information Processing Society. Nafips '07 Meeting of the North American*. San Diego: IEEE, 2007, pp.69-72.
 - [8] Khoshgoftaar T M, Gao K, Seliya N. Attribute Selection and Imbalanced Data: Problems in Software Defect Prediction. *Proceedings of the 22nd IEEE International Conference on Tools with Artificial Intelligence*, Arras: IEEE, 2010, pp.137-144.
 - [9] Kaminsky K, Boetticher G D. Better Software Defect Prediction Using Equalized Learning with Machine Learners. 2004, vol.16, no.4, pp.18-25.
 - [10] Guo S, Chen R, Li H, et al. Identify Severity Bug Report with Distribution Imbalance by CR-SMOTE and ELM, *International Journal of Software Engineering and Knowledge Engineering*, 2019, vol.29, no.2, pp.139-175.
 - [11] Moser R. A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction, *ACM/IEEE, International Conference on Software Engineering*. Leipzig: IEEE, 2009, pp.181-190.
 - [12] Siers M J, Islam M Z. Software defect prediction using a cost sensitive decision forest and voting, and a potential solution to the class imbalance problem. *Information Systems*, 2015, vol.51, pp.62-71.
 - [13] Miao L, Liu M, Zhang D. Cost-sensitive feature selection with application in software defect prediction, *International Conference on Pattern Recognition*. Tsukuba: IEEE, 2012, pp. 967-970.
 - [14] Arar O M, Ayan K. Software defect prediction using cost-sensitive neural network. *Applied Soft Computing*, 2015, vol.33, pp.263-27.